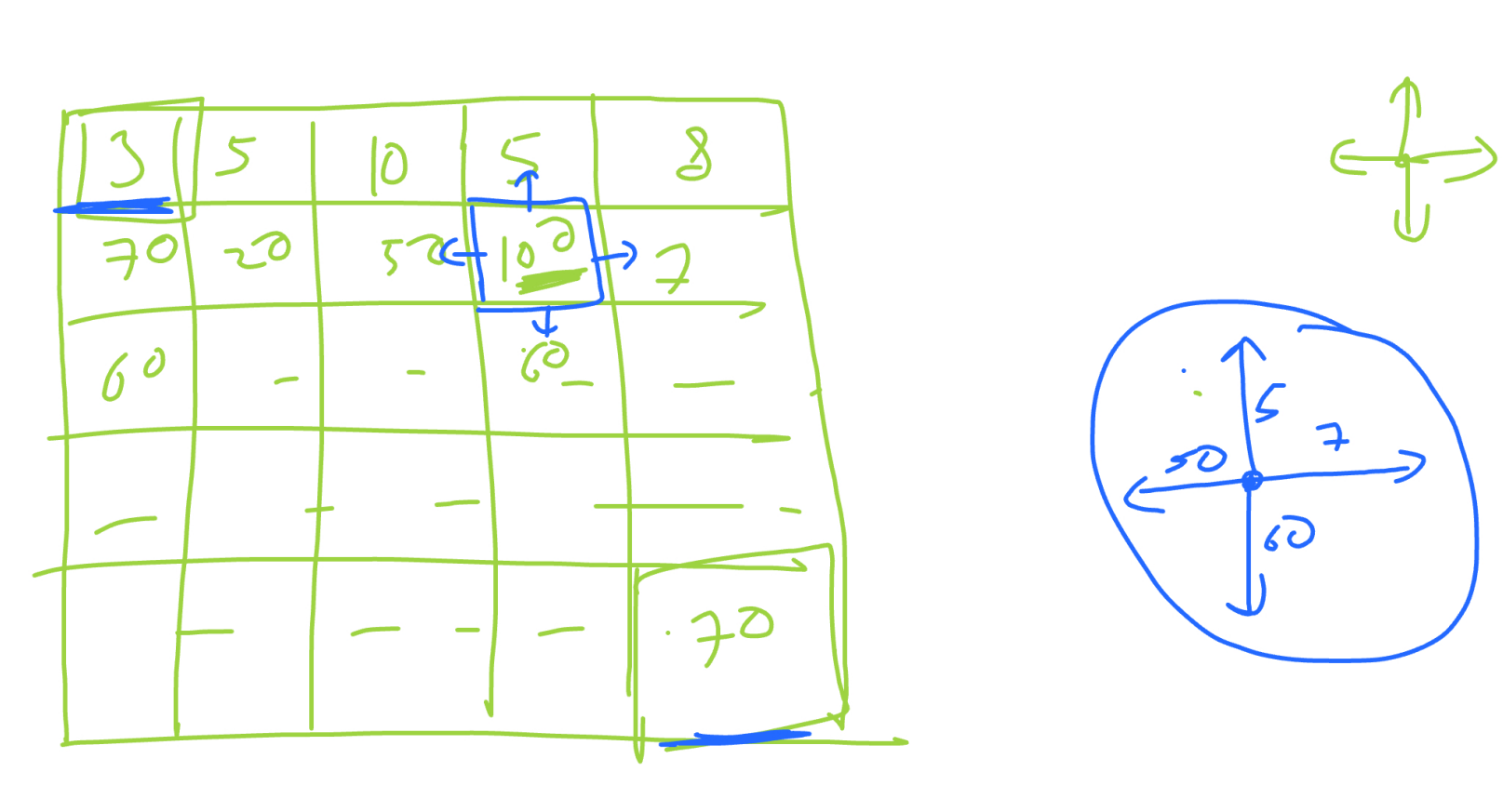
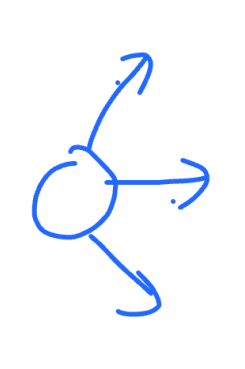
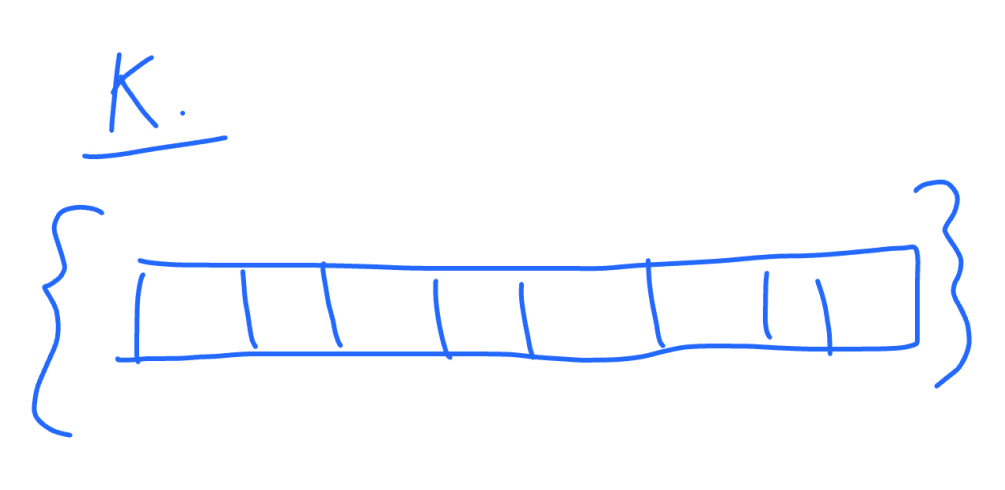






Dynamic Programming.

→ Simply the act of storing the results & using them later.

i.e., if you realise that you are calculating the same thing again & again.

→ Just calculate it once & store it & use the stored value later.

Repeated sub problems.

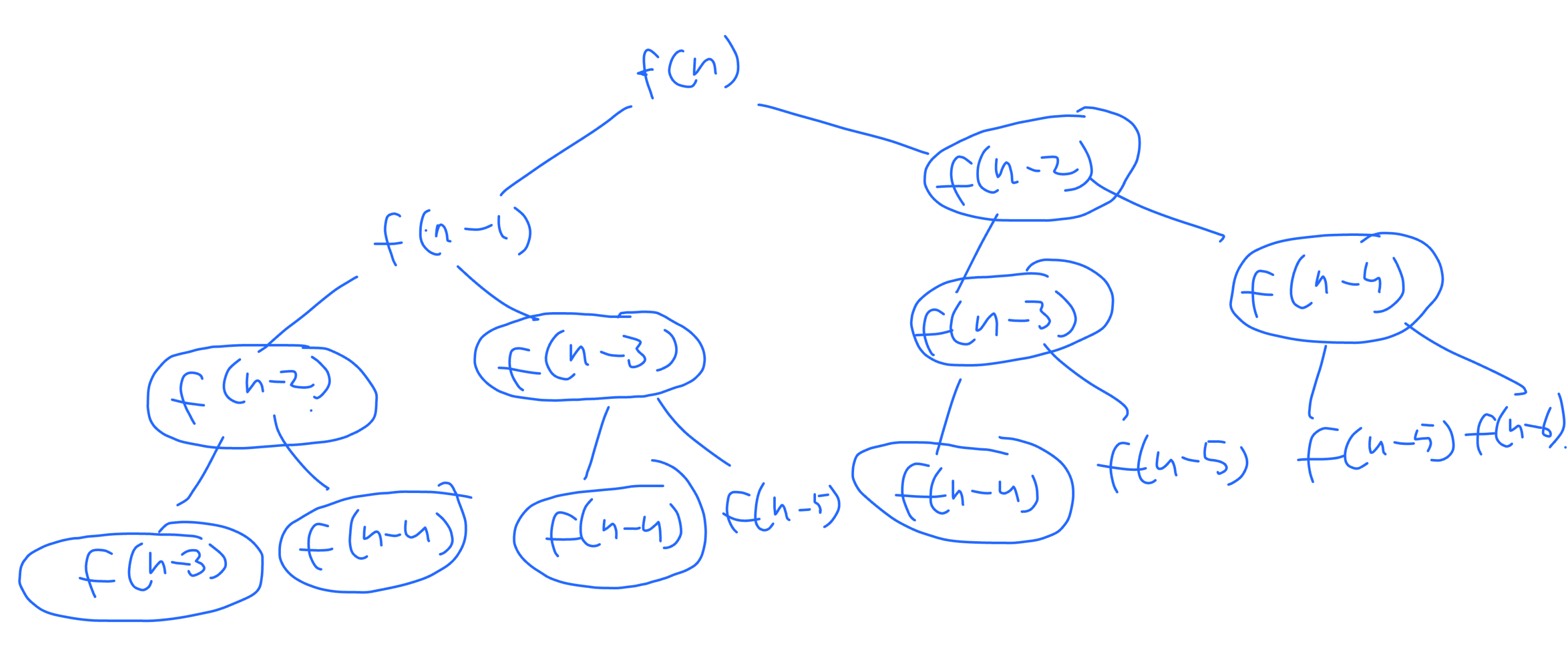
fib(n) = fib(n-1) + fib(n-2).

```
int fib(int n) {
 if(n ≤ 1) return n;

 return fib(n-1) + fib(n-2)
}
```

fib(0) = 0 fib(1) = 1

0, 1, 1, 2, 3, 5, 8, 13, 21 - - - -



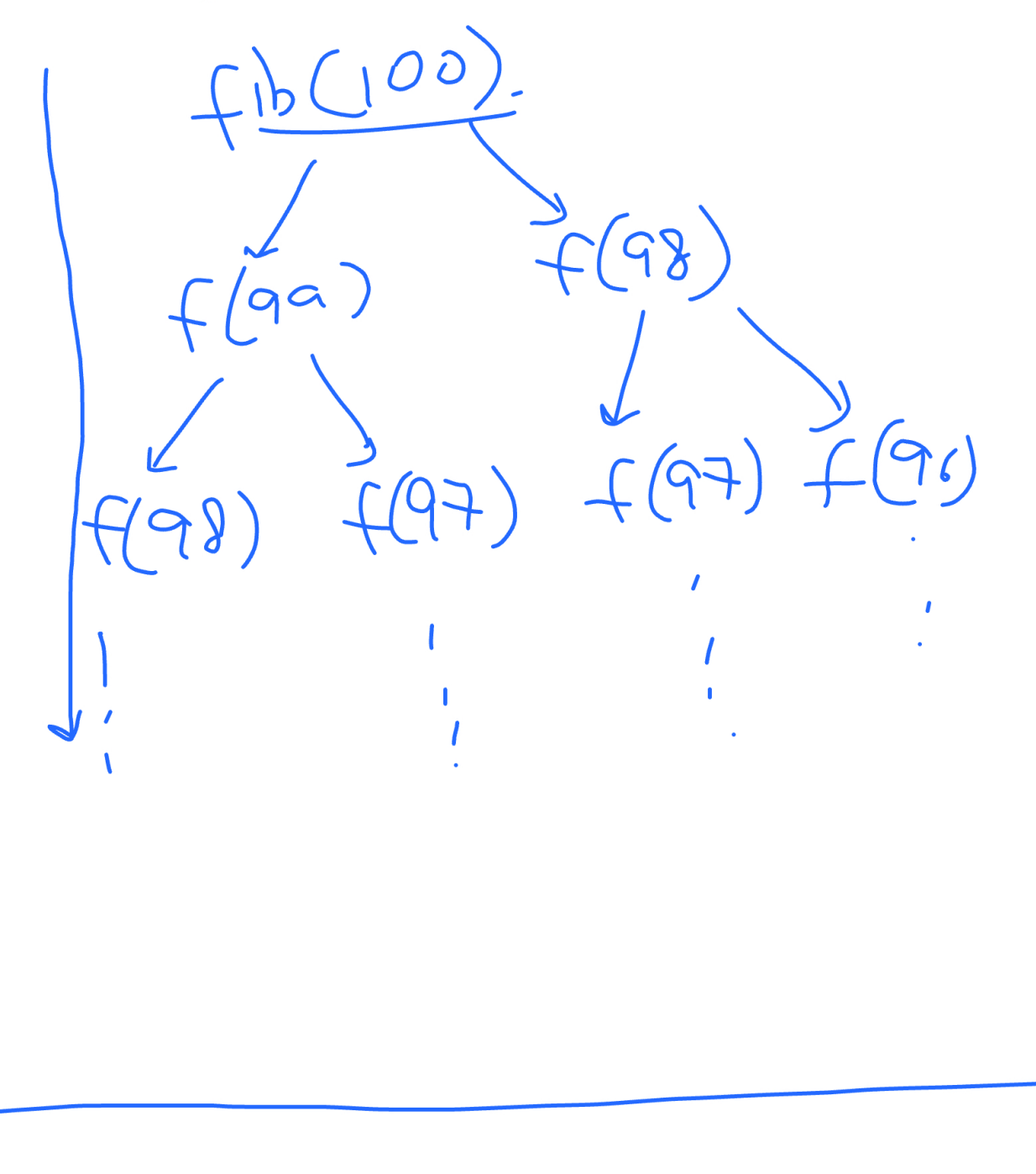
If there are repeated sub problems — its a clear indication that you can optimise your soln. using DP.

Ways to solve.

1) Top-Down DP / Memoization — Recursive in nature.

2) Bottom-Up DP / Tabulation — Iterative in nature.

Top-Down.



Bottom-Up.

f(100)

0, 1, 1, 2, 3, 5, 8, 13, 21, 34 - - - $\boxed{\ }$

LCS.

HELLO → HE, HL, ELO, HLO, ~~LHO~~

S1 = ABCDGH.
S2 = AEDFHR
O/P = 3 (ADH)

AB
BA

LCS($\overset{\downarrow}{A}$BCDGH, $\overset{\downarrow}{A}$EDFHR)

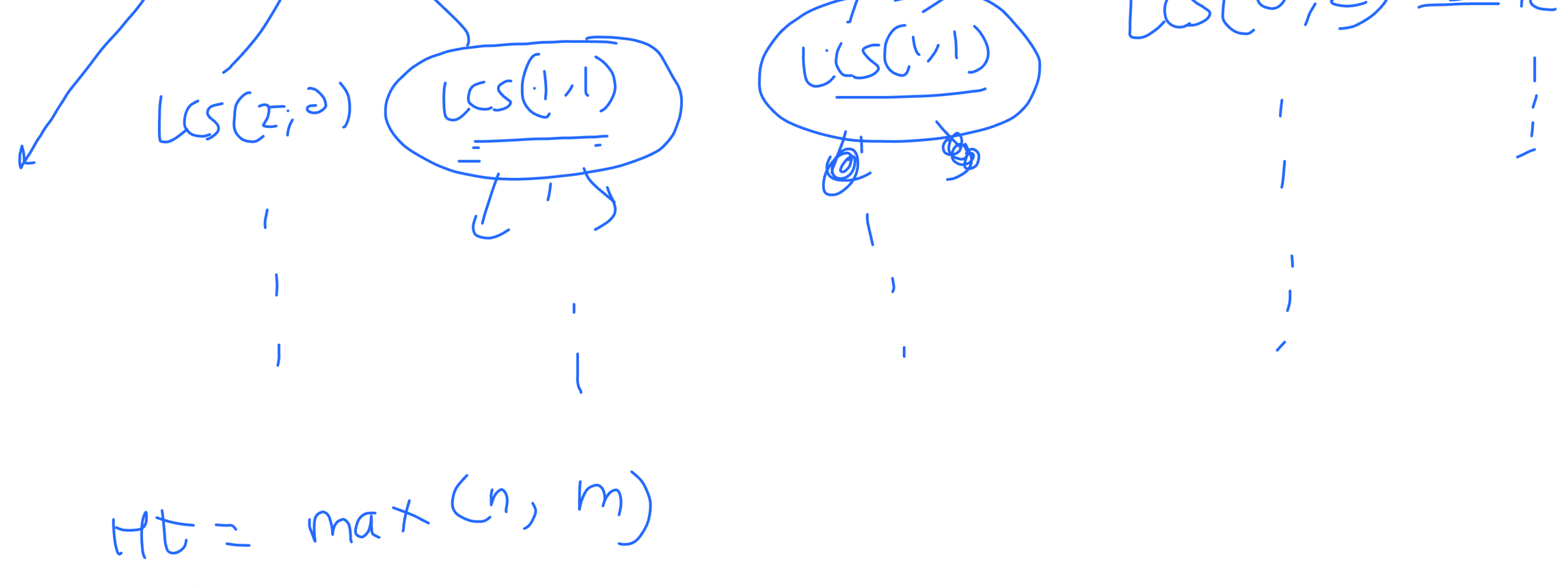
↓

1 + LCS($\overset{\downarrow}{B}$CDGH, $\overset{\downarrow}{E}$DFHR)

max [LCS(CDGH, EDFHR) , LCS(BCDGH, DFHR)]

$$LCS(i, j) \Rightarrow \begin{cases} 1 + LCS(i+1, j+1) & \text{if } s1[i] = s2[j] \\ \max\begin{pmatrix} LCS(i+1, j), \\ LCS(i, j+1) \end{pmatrix} & \text{otherwise.} \end{cases}$$

LCS("", "ABC") = 0



Ht = max(n, m)

$$2^0c + 2^1c + 2^2c + 2^3c - - - - + 2^{H-1}c = O(2^H)$$
$$= O(2^{\max(n,m)}).$$

DP.

TC:

1) Find the no. of unique recursive states (It is always equal to the no. of values you store in the memo table) → = x

2) Find the work done at each node → = y.

3) TC = O(x y).

		A	E	D	F	M	R
	0	0	0	0	0	0	0
1 A(0)	0	1	1	1	1	1	1
B	0	1	1	1	1	1	1
C	0	1	1	1	1	1	1
D	0	1	1	2	2	2	2
G	0	1	1	2	2	2	2
M	0	1	1	2	2	3	3

← result.

dp[i][j] = LCS(s1[0 .... i-1], s2[0 .... j-1])

$$LCS(\overset{\downarrow}{A}, A\overset{\downarrow}{E}) = \max\begin{pmatrix} LCS("", AE) \\ LCS(A, A) \end{pmatrix}$$

$$dp[i][j] = \max\begin{pmatrix} dp[i-1][j], \\ dp[i][j-1] \end{pmatrix} \Bigg] \text{ when not same.}$$

$$dp[i][j] = 1 + dp[i-1][j-1] \Big] \text{ when same.}$$

0-1 Knapsack.

vals: [60, 100, 120].
wts: [10, 20, 30].
W = 50

res: ~~120~~ 220
w = ~~50~~ ~~20~~ 0

O/P: 220

vals: [60 100 120]
wts: [10 20 50]
W = 50

W = 50

res: ~~120~~
w = ~~50~~ 0

res: ~~60~~ ~~60~~ 160
w = ~~50~~ ~~40~~ 20

O/P: 160

maxVal(i, w) —leave it→ maxVal(i+1, w)

maxVal(i, w) —take it→ vals[i] + maxVal(i+1, w - wts[i])

→ return max.